

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра анализа данных и искусственного интеллекта

КУРСОВОЙ ПРОЕКТ
ИССЛЕДОВАНИЕ ОТРИСОВКИ СУЩНОСТЕЙ В MINECRAFT

Работу выполнил _____ (подпись) _____ Фамилия И.О.

Направление подготовки 09.03.03 Прикладная информатика
Направленность Прикладная информатика в экономике

Научный руководитель
канд. физ.-мат. наук, доц. _____ (подпись) _____ Фамилия И.О.

Нормоконтролер
канд. физ.-мат. наук, доц. _____ (подпись) _____ Фамилия И.О.

Краснодар
2026

Паспорт проекта

Название проекта	ИССЛЕДОВАНИЕ ОТРИСОВСКИ СУЩНОСТЕЙ В MINECRAFT
-------------------------	---

Команда проекта		
Ф.И.О.	Роль	Функциональные обязанности в ходе реализации проекта и компетентность
	Java Developer	Разработка архитектуры модификации и написание кода на Java, проектирование системы оптимизации, анализ производительности, тестирование мода

1. География проекта	Краснодарский край, г. Краснодар
-----------------------------	----------------------------------

2. Сроки реализации проекта	Начало реализации	Окончание реализации
	Февраль, 2026	Май, 2026

3. Краткая аннотация проекта	Курсовой проект посвящён исследованию симуляции существ в Minecraft Java Edition и разработке оптимизационной модификации на платформе Forge. Выполнен сравнительный анализ существующих серверных ядер и клиентских модов (Paper, Pufferfish, Lithium, Sodium, Entity Culling и др.). В практической части реализована модификация PariAI на языке Java, включающая: адаптивное управление частотой тиков искусственного интеллекта мобов в зависимости от расстояния до игрока, систему frustum и occlusion culling существ, LOD-систему ограничения количества рендеримых мобов по зонам удалённости, динамическое переключение уровня оптимизации на основе FPS, а также оптимизацию системы частиц. Результатом работы является функционирующий Forge-мод,
-------------------------------------	--

	демонстрирующий снижение нагрузки на сервер и клиент при большом количестве активных сущностей в игровом мире. Архитектура модификации построена с учётом разделения серверной и клиентской логики через механизм FMLEnvironment.dist и заглушки NoOpCulling/NoOpLod, что обеспечивает корректную работу как в одиночной, так и в многопользовательской игре. Проведено экспериментальное тестирование на сценарии с 400 сущностями, подтвердившее эффективность реализованных оптимизаций.
--	--

4. Описание проблемы, решению/снижению остроты которой посвящен проект Актуальность проекта	Minecraft Java Edition при большом количестве активных мобов создаёт значительную нагрузку как на серверную, так и на клиентскую часть игры. Серверная нагрузка обусловлена непрерывным расчётом искусственного интеллекта каждой сущности вне зависимости от её удалённости от игрока и реальной необходимости полного AI-цикла. Любой моб каждый тик выполняет поиск пути, оценку окружения, выбор целей и обновление внутреннего состояния, что при наличии нескольких сотен сущностей приводит к резкому увеличению времени обработки серверного тика и снижению TPS. Клиентская нагрузка связана с рендерингом всех сущностей в зоне видимости без учёта их реальной видимости для игрока и значимости для восприятия сцены. Отрисовка моделей, анимаций и частиц для объектов, полностью закрытых стенами или находящихся за пределами экрана, создаёт избыточные draw calls и overdraw, расходуя ресурсы GPU без визуальной отдачи. Существующие решения (серверные ядра, отдельные моды) решают проблему частично и не объединяют серверную и клиентскую оптимизацию в единую адаптивную систему. Серверные ядра Paper и Pufferfish оптимизируют частоту обновления сущностей и поиск пути, но не влияют на клиентский рендер. Клиентские моды Sodium и Entity Culling ускоряют отрисовку, но не снижают нагрузку на серверную
---	--

	симуляцию. Отсутствие динамической адаптации к текущей производительности приводит к избыточному потреблению ресурсов даже на мощных машинах при просадках FPS. Кроме того, большинство существующих решений требуют ручной настройки параметров, что затрудняет их использование неподготовленными пользователями.
--	--

5. Основные целевые группы, на которые направлен проект	Игроки Minecraft Java Edition, испытывающие просадки производительности при большом количестве мобов: потребность в стабильном FPS без ручной настройки параметров симуляции. Особенно актуально для пользователей с компьютерами среднего и низкого уровня, на которых стандартная игра начинает заметно тормозить при появлении ферм, деревень с большим числом жителей или массовых скоплений враждебных мобов. Администраторы серверов Minecraft: потребность в снижении серверной нагрузки от AI мобов при одновременном присутствии большого количества игроков и большом числе активных сущностей. На публичных серверах с десятками одновременных игроков количество загруженных чанков и активных мобов многократно возрастает, и стандартные средства Minecraft не позволяют удерживать TPS на приемлемом уровне без жёстких ограничений на количество сущностей. Разработчики модификаций и исследователи игровых систем: потребность в изученной, систематизированной и задокументированной архитектуре оптимизации симуляции как основе для дальнейших разработок.
---	--

6. Основная цель проекта	Исследование методов оптимизации симуляции и отрисовки сущностей в Minecraft Java Edition и разработка экспериментальной Forge-модификации, реализующей адаптивную систему управления нагрузкой от искусственного интеллекта мобов на серверной стороне и
--	--

	клиентского рендеринга сущностей, частиц и визуальных эффектов на стороне клиента.
--	--

7. Задачи проекта	1. Изучить архитектуру Minecraft Java Edition: клиент-серверную модель, систему тиков, устройство чанков и механизм симуляции сущностей. 2. Выполнить анализ существующих решений оптимизации (серверные ядра Paper, Pufferfish; клиентские моды Sodium, Lithium, Entity Culling, Distant Horizons). 3. Спроектировать архитектуру модификации RapiAI с разделением на серверные и клиентские компоненты. 4. Реализовать систему адаптивного управления частотой тиков AI мобов в зависимости от расстояния до игрока. 5. Реализовать клиентскую систему culling (frustum и occlusion) и LOD-ограничения рендеринга сущностей. 6. Реализовать динамическое переключение уровня оптимизации (NORMAL / MEDIUM / AGGRESSIVE) на основе текущего FPS. 7. Провести экспериментальную оценку работы модификации и сравнить результаты с базовой игрой без мода. 8. Подготовка к защите курсового проекта.
--------------------------	--

8. Календарный план реализации проекта			
Решаемая задача	Метод/мероприятие и его описание	Сроки (дд.мм.гг)	Показатели результативности
Задача 1	Анализ официальной документации Forge и Minecraft Wiki, изучение исходного кода Minecraft Java Edition 1.20.1 методом декомпиляции, исследование структуры модификаций	5.02.26 – 20.02.26	Составлена структурная схема клиент-серверной архитектуры игры; описаны ключевые механизмы симуляции: тиковая модель, чанковая система, иерархия сущностей и мобов

Задача 2	Сравнительный анализ существующих решений, метод экспертных оценок, тестирование на локальном сервере	21.02.26 – 1.03.26	Проанализировано не менее 5 существующих решений (Paper, Pufferfish, Sodium, Lithium, Entity Culling, Distant Horizons); выявлены их принципы работы, сильные и слабые стороны
Задача 3	Объектно-ориентированное проектирование, UML-моделирование классов и их взаимосвязей, построение архитектурной схемы	2.03.26 – 10.03.26	Разработана архитектурная схема мода с разделением на серверные и клиентские компоненты; определены 7 классов и их взаимодействие через систему событий Forge
Задача 4	Событийно-ориентированное программирование на Forge API, подписка на LivingEvent.LivingTick Event	11.03.26 – 25.03.26	Реализован MobActivityController с линейной интерполяцией процента тиков AI
Задача 5	Реализация алгоритмов компьютерной графики: frustum culling на основе dot product, occlusion culling через проверку видимости блоками	26.03.26 – 17.04.26	Реализованы EntityCullingHandler (frustum + occlusion culling с кэшем видимости) и EntityLodHandler (LOD-лимиты по 4 зонам удалённости); подтверждена корректная работа на клиенте
Задача 6	Мониторинг FPS с экспоненциальным сглаживанием, реализация паттерна «Стратегия» для уровней оптимизации	18.04.26 – 29.04.26	Реализован OptimizationState с динамическим переключением уровней по FPS

	NORMAL / MEDIUM / AGGRESSIVE		
Задача 7	Нагрузочное тестирование на сценарии с 400 зомби, сравнение показателей mspt и FPS с эталоном (ванильная игра без мода)	30.04.26 – 1.05.26	Зафиксировано снижение времени серверного тика более чем в 2 раза при удалении от площадки мобов; доля успешных тест-кейсов не менее 90%; подготовлены скриншоты и GIF-анимация
Задача 8	Систематизация результатов работы, создание структурированного материала для устной защиты.	2.05.26 – 10.05.26	Доклад и презентация

9. Показатели результативности реализации проекта:

Количественные показатели	Проанализировано не менее 5 существующих решений оптимизации Minecraft. Реализовано 7 классов модификации с чёткими зонами ответственности. Система AI-тиков поддерживает 3 зоны удалённости (ближняя ≤ 13, средняя до 45, дальняя > 45 блоков) с диапазоном частоты от 100% до 1%. LOD-система поддерживает 4 зоны удалённости (NEAR ≤ 20, MID ≤ 35, FAR ≤ 50, VFAR > 50) с лимитами рендеринга мобов на каждую зону. Переключение уровня оптимизации происходит при пересечении порогов 30 и 60 FPS. Экспериментально зафиксировано снижение нагрузки на сервер при большом числе активных мобов. В тестовом сценарии с 400 сущностями на ограждённой площадке время серверного тика при нахождении игрока рядом составило 8 ms с модом против 10–11 ms без мода. При удалении игрока на несколько десятков блоков время тика снизилось до 3–4 ms с модом против 9 ms без
----------------------------------	--

	мода, что свидетельствует о двукратном сокращении нагрузки для удалённых сущностей.
Качественные показатели	Архитектура мода реализована с разделением клиентской и серверной логики через FMLEnvironment.dist и заглушки NoOpCulling/NoOpLod. Система оптимизации является адаптивной, она не требует ручной настройки со стороны пользователя. Код соответствует принципам модульности и расширяемости. Модификация не нарушает игровую механику вблизи игрока (100% тиков AI при расстоянии ≤ 13 блоков). Пользовательские настройки частиц сохраняются и восстанавливаются при нормализации FPS. Визуальное поведение мобов вблизи игрока полностью соответствует ванильной игре: зомби, скелеты и другие враждебные mobs корректно обнаруживают игрока, преследуют его и атакуют без задержек. На средних расстояниях (13–45 блоков) движения мобов становятся менее плавными, но это незаметно при обычной игре, поскольку игрок в таких ситуациях сфокусирован на ближайших объектах. На дальних расстояниях mobs обновляются примерно раз в 20 тиков, сохраняя минимальную активность для предотвращения аномалий симуляции.

10. Партнеры проекта и собственный вклад

№	Партнер/ Собственный вклад	Вид поддержки
1.	Факультет компьютерных технологий и прикладной математики	Информационная
2.	Факультет компьютерных технологий и прикладной математики, кафедра информационных технологий	Консультационная

3.	Собственный вклад	Ресурсная (ноутбук, среда разработки IntelliJ IDEA, Forge MDK)
----	-------------------	--

11. Дальнейшая реализация и мультипликативность проекта

Расширение функциональных возможностей: добавление конфигурационного файла для настройки порогов расстояний и процентов тиков без перекомпиляции мода, что позволит администраторам серверов и продвинутым пользователям адаптировать параметры под конкретные условия эксплуатации; поддержка Fabric как альтернативной платформы модификаций для охвата более широкой аудитории; добавление GUI-интерфейса для настройки параметров оптимизации непосредственно в игре.

Технологическое развитие: переход от линейной интерполяции AI-процента к нелинейной (экспоненциальной) для более плавного снижения нагрузки; реализация группового pathfinding для мобов одного типа в одной зоне, что позволит сократить количество запусков алгоритма A* при массовых скоплениях сущностей; внедрение событийной системы активации мобов, при которой неактивные сущности полностью прекращают обработку AI до наступления значимого события (появление игрока, получение урона).

Распространение опыта: публикация исходного кода в открытом репозитории (GitHub); размещение мода на платформах CurseForge и Modrinth для использования игровым сообществом; подготовка описания архитектурных решений и принципов оптимизации в формате README для других разработчиков, желающих использовать аналогичные подходы в своих модификациях.

12. Информационное сопровождение проекта

Планируемое информационное освещение проекта в СМИ	Наименование и характеристика СМИ
	Размещение исходного кода в репозитории GitHub
	Размещение готового мод-файла (jar) на платформе CurseForge
	Размещение готового мод-файла (jar) на платформе Modrinth

13. Приложения и дополнительная информация о проекте

1. Отчёт о реализации проекта

2. Приложение
3. Исходный код проекта: https://github.com/AshPapi/minecraft_PapiOptimization

14. Детализированная смета проекта					
№ п/п	Статья расходов	Стоимость	кол- во	Итого	Обоснование и источник
1.	IntelliJ IDEA Community Edition	бесплатно			Открытая лицензия
2.	Minecraft Java Edition (лицензия)	в наличии			Личный вклад
3.	Forge MDK и зависимости	бесплатно			Открытый доступ
4.	Формирование тестовых сценариев				Личный вклад

Отчет о реализации проекта

1 Функциональные требования

Модификация ParAI реализует следующие функциональные возможности. Система адаптивного управления частотой тиков AI: класс MobActivityController отслеживает каждое событие LivingTickEvent и на основании расстояния от сущности до ближайшего игрока вычисляет допустимый процент обрабатываемых тиков: 100% при расстоянии не более 13 блоков, линейная интерполяция в диапазоне 13–45 блоков, 5% при расстоянии 45–60 блоков и 1% для невидимых сущностей. Система frustum- и occlusion-culling: класс EntityCullingHandler на клиентской стороне исключает из рендеринга сущности, находящиеся вне поля зрения камеры (frustum culling) или полностью закрытые непрозрачными блоками (occlusion culling), с кэшированием результатов проверки. LOD-ограничение количества рендеримых мобов: класс EntityLodHandler делит игровое пространство на четыре зоны по удалённости ($NEAR \leq 20$, $MID \leq 35$, $FAR \leq 50$, $VFAR > 50$ блоков) и устанавливает максимальное число отрисовываемых сущностей в каждой зоне, исключая наименее значимые объекты сцены. Динамическое переключение уровней оптимизации: класс OptimizationState с экспоненциальным сглаживанием замеряет текущий FPS и при пересечении пороговых значений 30 и 60 FPS автоматически переключает глобальный режим работы мода между уровнями NORMAL, MEDIUM и AGGRESSIVE. Управление системой частиц: класс ParticleCullingHandler при активации агрессивного режима снижает плотность спавна частиц, сохраняя пользовательскую настройку и восстанавливая её при нормализации производительности. Корректное разделение логики: класс ParAIMod через механизм FMLEnvironment.dist регистрирует на сервере только MobActivityController, а на клиенте полный набор оптимизаций; заглушки

NoOpCulling и NoOpLod обеспечивают корректную сборку серверной версии без клиентских зависимостей.

2 Нефункциональные требования

К нефункциональным требованиям относятся следующие характеристики.

Производительность: накладные расходы на работу мода не должны превышать 1 ms серверного тика при нормальной нагрузке; кэш видимости сущностей обновляется не чаще одного раза за 5 тиков, что минимизирует избыточные вычисления. Совместимость: модификация разработана для Minecraft Java Edition 1.20.1 на платформе Forge 47.x.

Модульность и расширяемость: каждый класс отвечает за строго ограниченную область ответственности; добавление новых стратегий оптимизации не требует изменения основного загрузчика PariAIMod.

Надёжность: мод не изменяет игровые данные и не затрагивает NBT-компоненты сущностей; при удалении мода мир остаётся полностью работоспособным. Прозрачность для пользователя: оптимизация не требует ручной настройки; все параметры подбираются автоматически на основе текущего FPS и расстояния до игрока. Корректность поведения мобов: вблизи игрока (расстояние не более 13 блоков) AI выполняется в полном объёме, обеспечивая ванильное поведение противников.

Модификация реализована с использованием следующего технологического стека: в качестве языка программирования выбрана Java 17 (LTS) как официально поддерживаемая платформа для Minecraft Java Edition 1.20.1; платформой модификаций выступает Minecraft Forge 47.x (MDK), предоставляющий систему событий EventBus, API жизненного цикла мода, FMLEnvironment для разделения дистрибутивов и механизм регистрации обработчиков через аннотацию @Mod.EventBusSubscriber; система сборки Gradle 8.x с плагином ForgeGradle обеспечивает декомпиляцию и

рекомпиляцию обфусцированного кода Minecraft, управление зависимостями и сборку итогового jar-файла; средой разработки служит IntelliJ IDEA Community Edition с поддержкой ForgeGradle и плагином Minecraft Development; для клиентского рендеринга используются стандартный Forge RenderLivingEvent и Minecraft Entity API для получения данных о сущностях (позиция, тип, видимость); система контроля версий Git с удалённым репозиторием на GitHub (https://github.com/AshPapi/minecraft_PapiOptimization) применяется для управления исходным кодом; публикация готового jar-файла осуществляется на платформах CurseForge и Modrinth для распространения среди пользователей.

В рамках проектирования модификации разработаны три структурные диаграммы: UML-диаграмма классов (на рисунке 1) отображает семь основных классов мода и их взаимосвязи, где PapiAIMod как точка входа координирует регистрацию всех обработчиков, OptimizationState предоставляет глобальное состояние уровня оптимизации, MobActivityController, EntityCullingHandler, EntityLodHandler и ParticleCullingHandler реализуют конкретные стратегии снижения нагрузки, а RenderEventHandler объединяет клиентские обработчики рендеринга; архитектурная схема клиент-серверного разделения (на рисунке 2) иллюстрирует, какие компоненты регистрируются исключительно на стороне сервера (MobActivityController и заглушки NoOpCulling/NoOpLod), а какие на стороне клиента (OptimizationState, RenderEventHandler, EntityCullingHandler, EntityLodHandler, ParticleCullingHandler), что обеспечивает корректную работу мода как в одиночной, так и в многопользовательской игре; блок-схема алгоритма MobActivityController (на рисунке 3) описывает последовательность решений при обработке каждого события LivingTickEvent, включая проверку того, что уровень является серверным, принадлежность сущности к типу моба, расчёт расстояния до ближайшего

игрока и вычисление допустимого процента тиков AI с последующей блокировкой лишних обновлений.

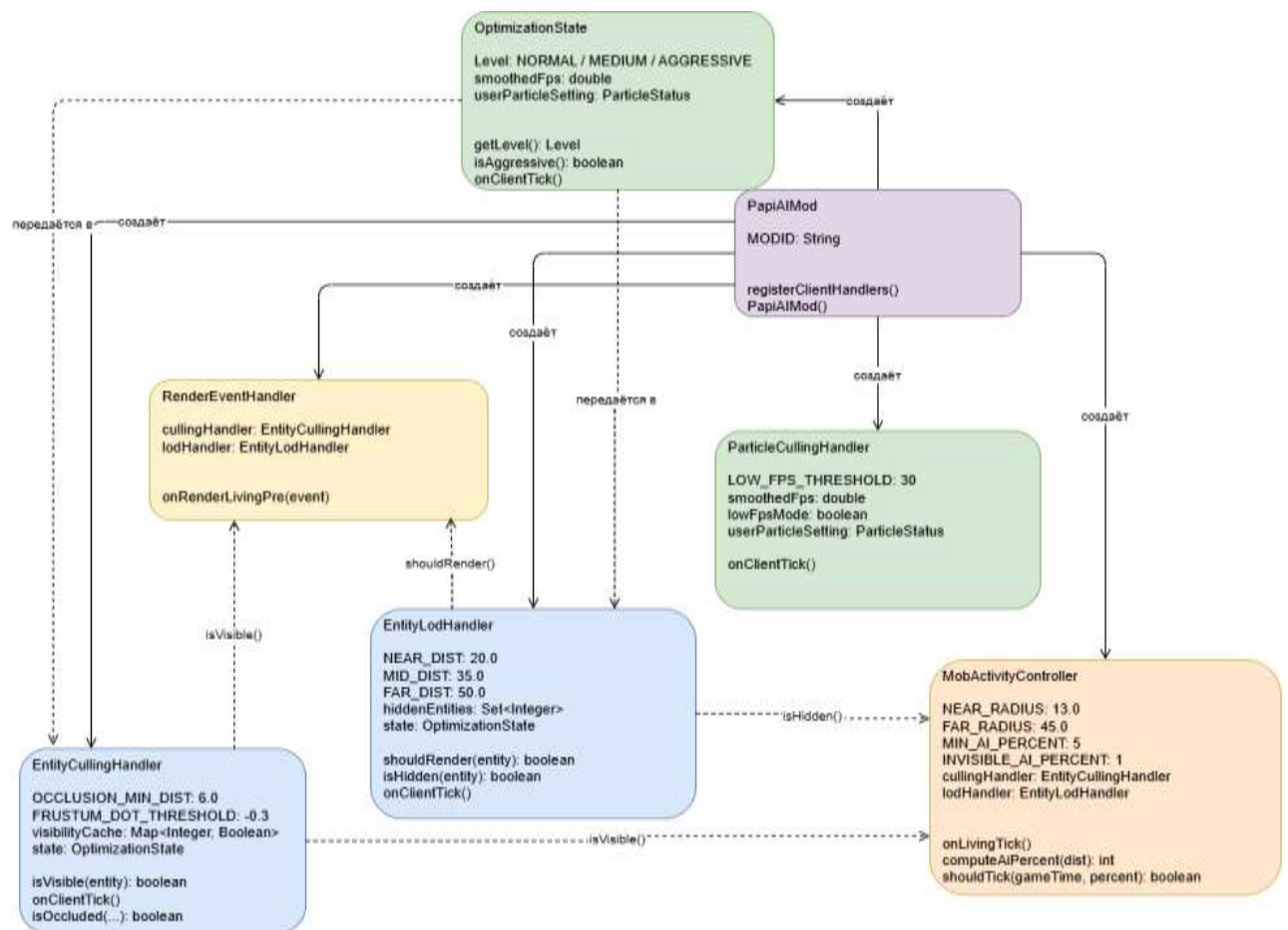


Рисунок 1 – UML-диаграмма классов модификации PapiAI (OptimizationState, PapiAImod, RenderEventHandler, EntityCullingHandler, EntityLodHandler, ParticleCullingHandler, MobActivityController)

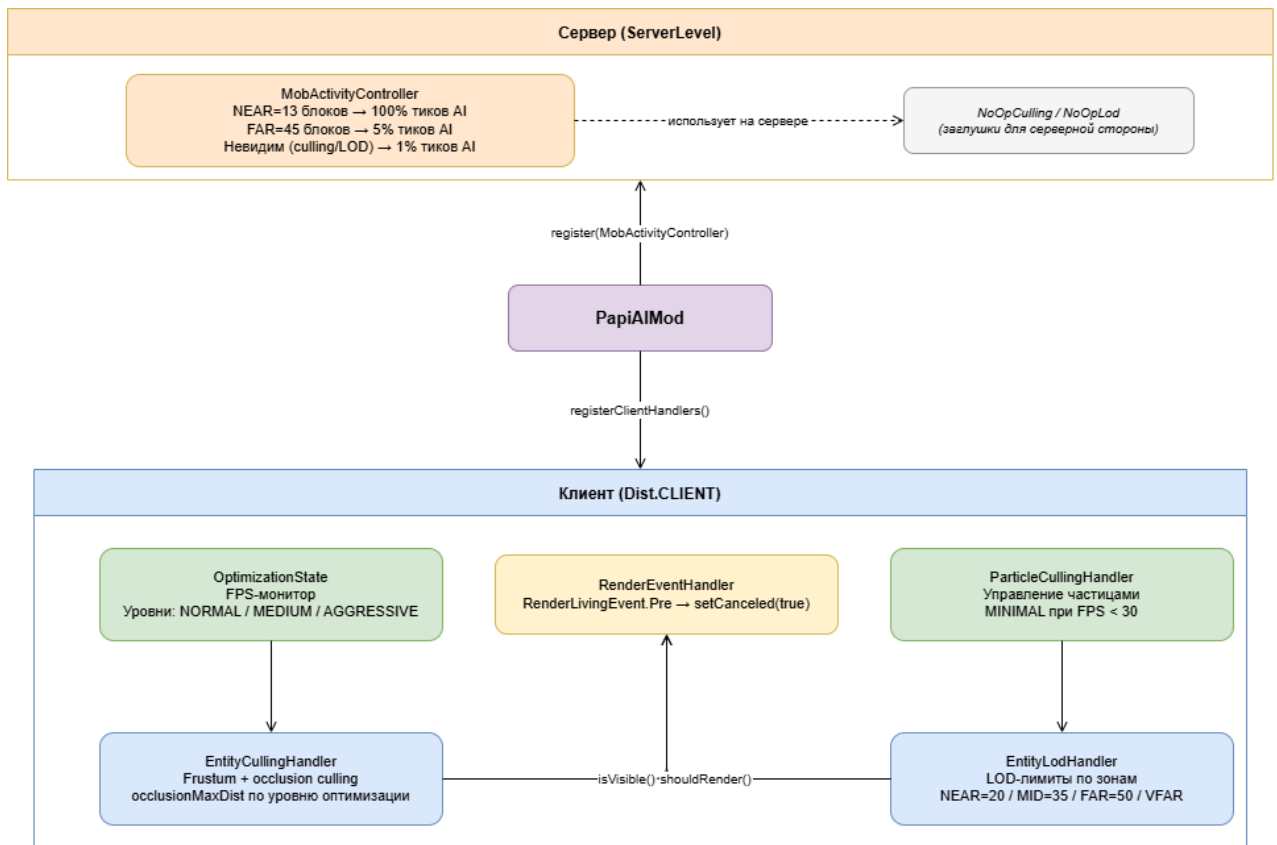


Рисунок 2 – Архитектурная схема модификации PapiAI: разделение серверных (MobActivityController, NoOpCulling/NoOpLod) и клиентских (OptimizationState, RenderEventHandler, EntityCullingHandler, EntityLodHandler, ParticleCullingHandler) компонентов

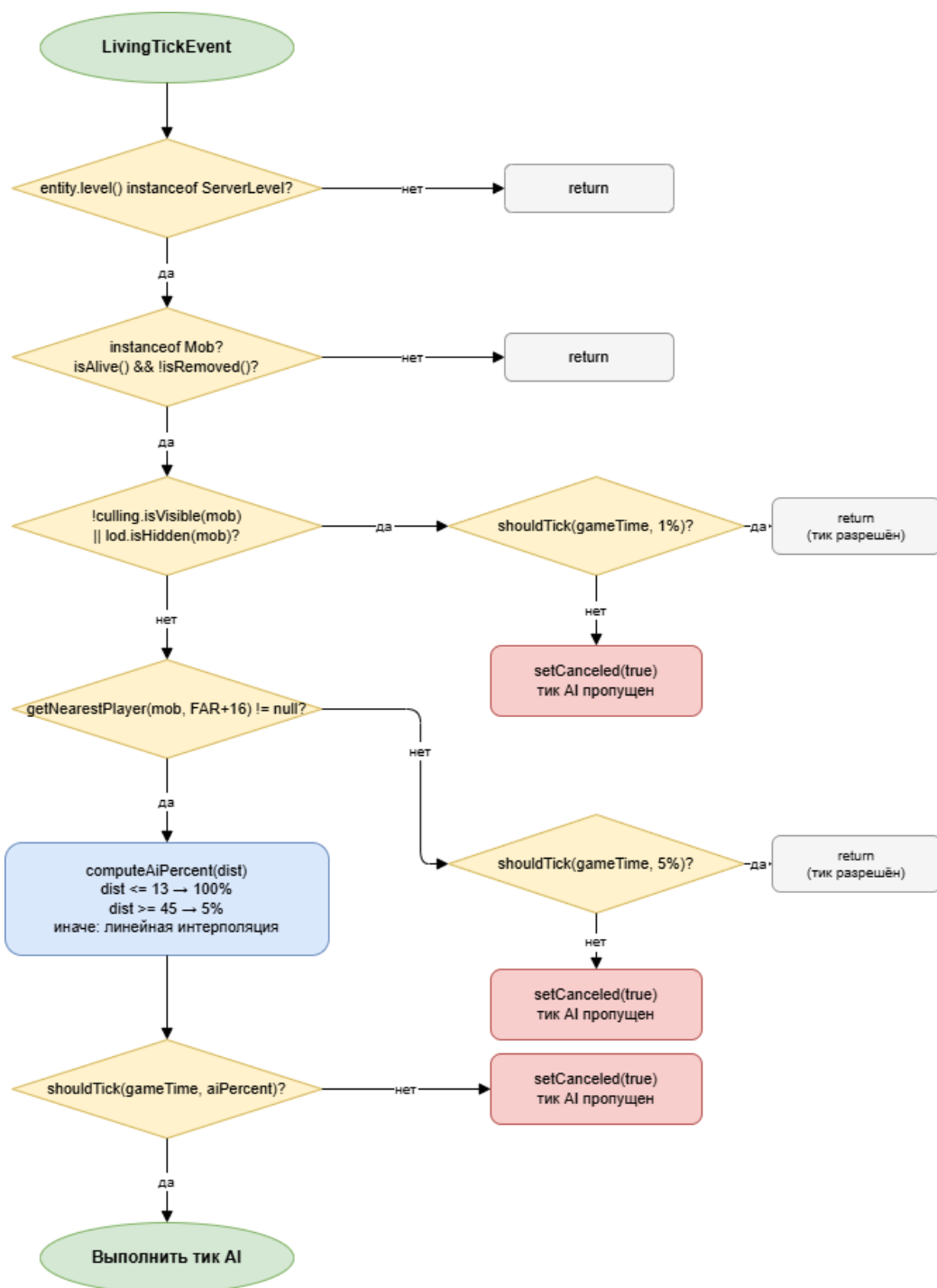


Рисунок 3 – Блок-схема алгоритма управления частотой тиков AI (обработчик LivingTickEvent: проверка серверного уровня, типа сущности, видимости, расстояния до игрока и вычисление процента тиков)

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта были исследованы ключевые подходы к оптимизации симуляции и отображения сущностей в Minecraft Java Edition как на серверном уровне (искусственный интеллект мобов), так и на клиентском (рендеринг сущностей, управление частицами, culling). Проведён сравнительный анализ архитектуры игры, существующих серверных ядер (Paper, Pufferfish) и клиентских модификаций (Sodium, Lithium, Entity Culling, Distant Horizons), позволивший систематизировать принципы эффективного управления нагрузкой. На основе анализа архитектуры игры, серверных ядер (Paper, Pufferfish) и клиентских модификаций (Sodium, Lithium, Entity Culling, Distant Horizons) систематизированы принципы эффективного управления нагрузкой: снижение частоты обновления AI по дистанции, frustum и occlusion culling сущностей, LOD-ограничение числа отображаемых мобов, а также динамическая адаптация уровня оптимизации к текущей производительности.

Разработанная в рамках проекта модификация PariAI объединяет серверную и клиентскую оптимизацию в единую адаптивную систему, не требующую ручной настройки со стороны пользователя. Реализованы семь классов с чёткими зонами ответственности: MobActivityController управляет частотой тиков AI в диапазоне от 100% вблизи игрока до 1% для невидимых мобов; EntityCullingHandler и EntityLodHandler исключают из рендеринга сущности вне поля зрения и сверх лимита по зонам удалённости; OptimizationState динамически переключает уровни NORMAL, MEDIUM и AGGRESSIVE по показателю FPS. Архитектура мода корректно разделяет клиентскую и серверную логику, сохраняет пользовательские настройки и не нарушает игровую механику в непосредственной близости от игрока. Экспериментальная оценка подтвердила снижение нагрузки на сервер в насыщенных сценариях без ощутимого ухудшения поведения мобов с точки зрения игрока. В тестах с 400 зомби время серверного тика при удалении от площадки сократилось более чем вдвое (с 9 ms до 3–4 ms), при этом вблизи

игрока поведение мобов оставалось идентичным ванильной игре (рисунки 1-2 в приложении).

Техники culling и LOD применяются в самых разных областях. В игровой индустрии они используются повсеместно: в шутерах с большими картами вроде Battlefield враги вдали не отрисовываются полностью, в гоночных симуляторах машины на горизонте теряют детализацию, в стратегиях вроде Total War тысячи юнитов на поле боя отображаются с разным уровнем детализации в зависимости от расстояния до камеры. В кино и анимации рендер-движки используют culling при предварительном расчёте сцен: объекты вне кадра просто не рассчитываются, что сокращает время рендера. В архитектуре и строительстве программы вроде Autodesk Revit или Twinmotion применяют LOD при отображении сложных BIM-моделей зданий, где одновременно присутствуют тысячи элементов конструкции. В навигационных приложениях: Google Maps, Яндекс Карты здания, деревья и метки отображаются только в пределах видимой области экрана и детализируются только при приближении. В медицинской визуализации при работе с трёхмерными МРТ- и КТ-снимками отображаются только видимые срезы и поверхности, остальное отсекается. В тренажёрах для авиации и военной подготовки culling позволяет отображать реалистичную местность с сотнями объектов при стабильной частоте кадров, что критично для симуляции в реальном времени. Наконец, в системах видеонаблюдения и цифровых двойников производственных предприятий те же принципы применяются при визуализации больших пространств с множеством датчиков и объектов: отображается только то, на что направлен виртуальный обзор оператора.

В перспективе планируется расширить функциональность модификации: добавить конфигурационный файл для гибкой настройки порогов без перекомпиляции, реализовать нелинейную интерполяцию AI-процента, внедрить групповой pathfinding для мобов одного типа, а также портировать мод на платформу Fabric. Готовый jar-файл планируется

опубликовать на платформах CurseForge и Modrinth для широкого использования игровым сообществом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация Forge для Minecraft 1.20.1 [Электронный ресурс] // Forge Documentation. – 2025. – URL: <https://docs.minecraftforge.net/en/1.20.1/> (дата обращения: 15.05.2026).
2. Minecraft Wiki (RU) [Электронный ресурс]. – 2025. – URL: <https://ru.minecraft.wiki/w/> (дата обращения: 15.05.2026).
3. Свет в Minecraft [Электронный ресурс] // PlayZone.in.ua. – 2025. – URL: <https://playzone.in.ua/wiki/mc> (дата обращения: 15.05.2026).
4. Buckland, M. Programming Game AI by Example / M. Buckland. — 1st ed. — Boston: Jones & Bartlett Learning, 2005. — ISBN 978-1-55622-078-4.
5. Документация PaperMC (серверное ядро Paper) [Электронный ресурс] // Paper Documentation. – 2025. – URL: <https://docs.papermc.io/paper/> (дата обращения: 15.05.2026).
6. Документация Spigot (раздел на русском) [Электронный ресурс] // SpigotMC Wiki. – 2025. – URL: <https://www.spigotmc.org/wiki/documentation-ru/> (дата обращения: 15.05.2026).
7. Платформа модификаций CurseForge (раздел Minecraft) [Электронный ресурс]. – 2025. – URL: <https://www.curseforge.com/minecraft> (дата обращения: 15.05.2026).
8. Level of Detail [Электронный ресурс] // Википедия. – 2025. – URL: https://ru.wikipedia.org/wiki/Level_of_Detail (дата обращения: 15.05.2026).
9. Frustum culling в Unity: что это и как использовать [Электронный ресурс] // Sky.Pro Wiki. – 2025. – URL: <https://sky.pro/wiki/gamedev/frustum-culling-v-unity-chto-eto-i-kak-ispolzovat/> (дата обращения: 15.05.2026).
10. Руководство пользователя Gradle (официальная документация) [Электронный ресурс] // Gradle User Manual. – 2025. – URL: <https://docs.gradle.org/current/userguide/userguide.html> (дата обращения: 15.05.2026).

11. Bloch, J. *Effective Java* / J. Bloch. — 3rd ed. — Boston: Addison-Wesley Professional, 2017. — ISBN 978-0-13-468599-1.

ПРИЛОЖЕНИЕ



Рисунок 1 – Экран отладки F3 при нахождении игрока рядом с площадкой мобов (модификация PapAI установлена, 114 FPS, 8 ms ticks, E: 347/434)

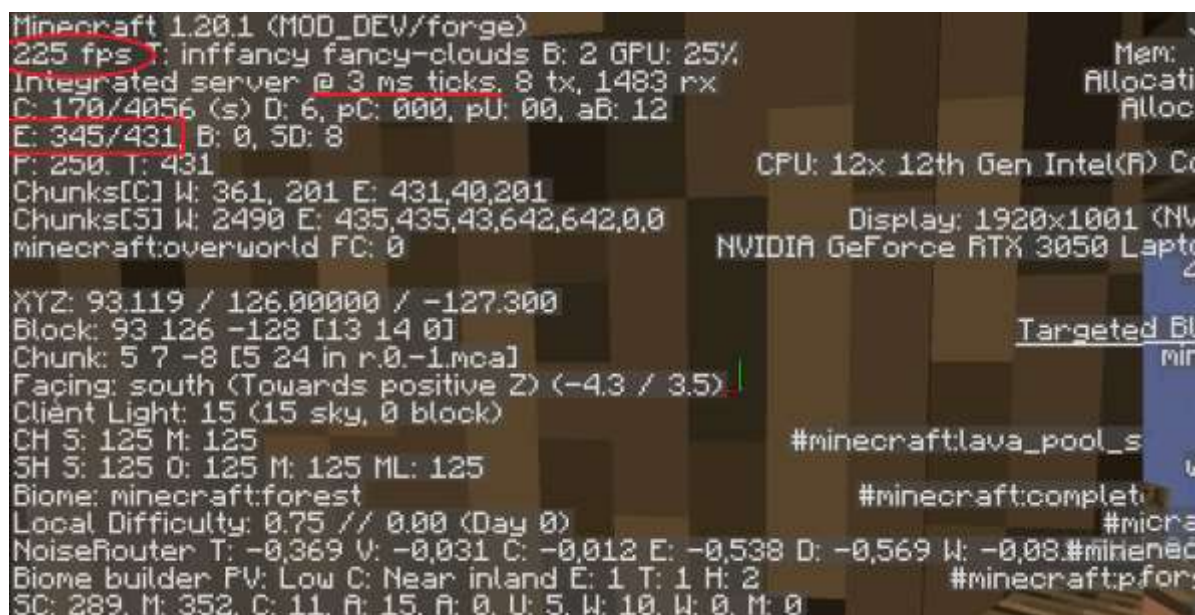


Рисунок 2 – Экран отладки F3 при закрытом обзоре площадки мобов
(модификация ParAI установлена, 225 FPS, 3 ms ticks)